AI Engineering Portfolio

Student Information Management System (SIMS) — AI-Assisted Laravel Development Project

Project: Student Information Management System (SIMS)
Course: Laravel Web Development
Student: Rexon Timbal
Date: November 1, 2026

A. AI Platform Used

Two roles of the same underlying model, **Claude (Anthropic)**, were used throughout the project, in two different front ends:

- **Claude (chat session)** — used as the planning and review layer. Every prompt sent to the coding agent was drafted and reasoned about here first, and every reported result was independently re-verified here afterward by reading the actual project files, routes, and database state.
- **Claude Code (VS Code extension)** — used as the execution layer, with direct terminal and file-system access inside the project. This is the agent that ran Artisan commands, wrote migrations, controllers, and views, and reported back what it did.

Purpose

Splitting planning from execution meant no single process both wrote the code and graded its own work. The chat session held the full picture of the handbook/rubric requirements and the running history of every step, while the VS Code agent focused narrowly on one module at a time with direct access to run and test real commands.

Strengths

- Direct terminal/file access in Claude Code meant migrations, Artisan commands, and file edits happened in one agentic loop instead of manual copy-paste of generated code.
- A separate reasoning session (chat) that never touched the filesystem directly made it a reliable, independent check — it could not accidentally rubber-stamp its own work.
- Both tools share the same underlying model, so prompt style and expectations carried over consistently between planning and execution with no translation loss.

Why It Was Selected

Claude Code was chosen specifically because this project's grading criteria (rubric section C) reward *verified* AI output, not just generated output. An agent with real execution access can be checked against ground truth (the database, the live app, the test suite) rather than trusted on its word alone — which shaped the whole verification habit documented throughout this portfolio.

B. Prompt History

Initial Prompt (Step 1 — Project Setup)

```
Create a new Laravel project in this current directory (do not create a nested
subfolder - install directly here).

Project requirements:
- Use Laravel (latest stable version compatible with my installed PHP).
- Configure the .env file to use MySQL as the database connection:
  - DB_CONNECTION=mysql
  - DB_HOST=127.0.0.1
  - DB_PORT=3306
  - DB_DATABASE=sims_db
  - DB_USERNAME=root
  - DB_PASSWORD=
- Create the MySQL database "sims_db" if it does not exist.
- Run the initial migrations (users, cache, jobs tables).
- After setup, run `php artisan route:list` to confirm the project is working.

This is the start of a Student Information Management System (SIMS) project.
```

Refined Prompt (Step 3 — Role-Based Authorization)

By Step 3, prompts had evolved from open-ended requests into fully-specified, numbered requirement lists with explicit verification instructions built in, rather than trusting a one-line request to produce the right result:

```
Add role-based authorization to the existing Breeze authentication system.

1. Add a migration to add a `role` column to the `users` table (string, default
   "staff", values limited to "admin"/"staff" via validation, not a DB enum).
2. Update the User model: add `role` to $fillable, add isAdmin() helper.
3. Create AdminMiddleware (alias: `admin`) that aborts 403 unless isAdmin().
4. Register the `admin` alias in bootstrap/app.php (Laravel 11/12 style).
5. Run the migration.
6. Create one admin user (name, email, password, role: "admin").
7. Show me `php artisan route:list` still works and confirm the admin user
   was created by querying it.
```

Debugging Prompt (Step 9 — REST API Route Collision)

After the REST API was scaffolded, the web Student module's navigation needed to be re-verified. The following line of investigation caught a real, non-obvious bug:

```
The Student web module's sidebar links may be affected by the new API routes.
Run `php artisan route:list` and check whether `Route::apiResource('students', ...)`
is colliding with the existing `Route::resource('students', ...)` route names.
If route('students.index') is no longer resolving to the web controller, fix the
API route names without changing the API's URIs.
```

Outcome: the agent found that Route::apiResource() defaulted to the same route names as the web resource (students.index, etc.), silently overriding route('students.index') used by the sidebar. Fixed with ->names('api.students'); URIs were unaffected. This is logged in full under *Prompt Iterations* below and in the Reflection Report as a key AI-output-verification example.

C. Prompt Iterations

Prompts were not one-shot. Several went through a visible initial → refined → verified cycle. Two representative examples:

Stage	Prompt / Action	Why it changed
Initial	“Add role-based authorization.”	Too vague to execute safely — no column spec, no default role, no middleware name.
Refined	Full 7-point numbered spec (migration shape, model helper, middleware name, registration point, seed data, verification command).	Removed ambiguity; gave the agent an explicit, checkable definition of done.
Verified	Queried the seeded user via Tinker directly from the chat session, read bootstrap/app.php to confirm the alias registration.	Confirmed the report matched reality instead of trusting the summary.

A second example is the route-authorization gap caught during self-review (Step 10): the chat session noticed that AdminMiddleware, built in Step 3, had never actually been applied to any route — every CRUD action only required auth, so Staff and Admin had identical access. The prompt was revised from "apply admin middleware to delete routes" to an explicit split (per-route ->except('destroy') plus a separate ['auth','admin'] group for the three destroy routes), specifically so that existing route(...) calls in the Blade views would not need to change. This was then verified behaviorally, not just by reading code: logged in as a Staff account, confirmed no Delete button rendered, then sent a raw DELETE request via fetch() directly (bypassing the UI) and confirmed a server-side **403 Forbidden**.

This demonstrates engineering thinking rather than simple prompt usage: prompts were revised in response to gaps found by independent review of the running system, not just because the AI's first draft produced an error.

D. Screenshots

Screenshots were captured during live browser verification at the following steps and should be inserted here before final submission (see PROJECT_LOG.md for the exact state each one confirms): Step 4 (dashboard layout/theme), Step 5 (Student create & list), Step 6 (Subject create), Step 7 (Enrollment create with populated dropdowns), Step 10 (Staff role — no Delete button, and the 403 response), Step 13 (login/register pages), Step 14 (delete confirmation modal and validation toasts).